

Variables et types de base

```
1 # Entiers (int)
2 a = 42
3 b = -7
4 c = 10**6      # 1_000_000
5
6 # Flottants (float)
7 pi = 3.14159
8 e = 2.71828e0
9
10 # Booléens (bool)
11 vrai = True
12 faux = False
13
14 # Chaînes de caractères (str)
15 nom = "Alice"
16 phrase = 'Python'
17 multiligne = """Plusieurs
18 lignes"""
```

Conversions de type

```
1 int("42")      # -> 42
2 float("3.14")  # -> 3.14
3 str(100)       # -> "100"
4 bool(0)        # -> False
```

Opérations mathématiques

```
1 # Opérateurs arithmétiques
2 a + b          # addition
3 a - b          # soustraction
4 a * b          # multiplication
5 a / b          # division (float)
6 a // b         # division entière
7 a % b          # modulo (reste)
8 a ** b         # puissance
9
10 # Fonctions intégrées
11 abs(x)         # valeur absolue
12 round(x, n)    # arrondi n décimales
13 pow(x, y)      # x**y
14 divmod(a, b)   # (quotient, reste)
```

Module math

```
1 import math
2 math.sqrt(2)    # racine carrée
3 math.exp(1)     # exponentielle
4 math.log(10)    # logarithme népérien
5 math.log10(100) # logarithme base 10
6 math.sin(math.pi/2) # sinus (radians)
7 math.factorial(5) # 5! = 120
8 math.gcd(12, 18) # PGCD = 6
9 math.comb(5, 2)  # coefficient binomial C(5,2)
```

Structures de données

Listes (mutables, ordonnées)

```
1 liste = [1, 2, 3, 4]
2 liste.append(5)      # ajout en fin
3 liste.insert(0, 0)   # insertion à l'index
4 liste.pop()          # retire le dernier
5 liste.pop(0)         # retire à l'index
6 liste.remove(3)       # retire la première occurrence
7 liste.extend([6,7])  # concaténation
8 liste.sort()         # tri en place
9 liste.reverse()      # inversion
10
11 # Compréhension de liste
12 carres = [x**2 for x in range(10)]
13 pairs = [x for x in range(20) if x % 2 == 0]
```

Tuples (immuables)

```
1 point = (2.0, -1.5, 3.7)
2 x, y, z = point      # unpacking
3 singleton = (42,)    # tuple à 1 élément
```

Dictionnaires (clé → valeur)

```
1 d = {"a": 1, "b": 2}
2 d["c"] = 3           # ajout/modification
3 v = d.get("a", 0)    # valeur ou 0 par défaut
```

```
4 d.keys()             # itérables sur les clés
5 d.values()           # itérables sur les valeurs
6 d.items()            # couples (clé, valeur)
7
8 # Compréhension de dictionnaire
9 carres = {x: x**2 for x in range(5)}
```

Ensembles (non ordonnés, éléments uniques)

```
1 s = {1, 2, 3}
2 s.add(4)
3 s.remove(2)
4 s2 = {3, 4, 5}
5 s | s2      # union
6 s & s2      # intersection
7 s - s2      # différence
```

Structures de contrôle

Conditions

```
1 if x > 0:
2     print("positif")
3 elif x == 0:
4     print("nul")
5 else:
6     print("négatif")
7
8 # Opérateur ternaire
9 signe = "positif" if x > 0 else "non positif"
```

Boucles for

```
1 for i in range(5):      # 0..4
2     print(i)
3
4 for i, val in enumerate(liste):
5     print(f"Indice {i}: {val}")
6
7 for cle, valeur in d.items():
```

```

8     print(cle, valeur)
9
10    # range(start, stop, step)
11    range(10)          # 0..9
12    range(1, 11)       # 1..10
13    range(0, 10, 2)    # 0,2,4,6,8

```

Boucles while

```

1  while condition:
2      # instructions
3      if autre_condition:
4          break      # sort de la boucle
5      if skip:
6          continue   # passe l'iteration suivante
7  else:
8      # ex cut si pas de break
9      pass

```

Fonctions

```

1  def nom_fonction(param1, param2=val_defaut):
2      """Docstring : description."""
3      # corps
4      return resultat
5
6  # Appel
7  res = nom_fonction(3, param2=5)
8
9  # Fonctions lambda (anonymes)
10  carre = lambda x: x**2

```

Arguments variables

```

1  def somme(*args):          # args est un
2      tuple
3      return sum(args)
4
5  def afficher(**kwargs):   # kwargs est un
6      dict
7      for cle, val in kwargs.items():
8          print(f"{cle} = {val}")

```

Récurtivité

```

1  def factorielle(n):
2      if n <= 1:             # cas de base
3          return 1
4      return n * factorielle(n-1)
5
6  # Attention la profondeur de recursion !
7  # sys.setrecursionlimit(3000)

```

Exemple : Fibonacci

```

1  def fibo(n, memo={}):     # memoisation
2      if n in memo:
3          return memo[n]
4      if n <= 1:
5          return n
6      memo[n] = fibo(n-1, memo) + fibo(n-2, memo)
7      return memo[n]

```

Programmation Orientée Objet

```

1  class MaClasse:
2      # Attribut de classe (partag )
3      compteur = 0
4
5      def __init__(self, valeur):
6          # Attribut d'instance
7          self.valeur = valeur
8          MaClasse.compteur += 1
9
10     def methode(self):
11         return self.valeur * 2
12
13     def __str__(self):
14         return f"Instance({self.valeur})"
15
16     def __repr__(self):
17         return f"MaClasse({self.valeur})"
18
19     # Surcharge d'op rateurs
20     def __add__(self, other):
21         return MaClasse(self.valeur + other.valeur)
22

```

```

23 # H ritage
24 class SousClasse(MaClasse):
25     def __init__(self, valeur, nom):
26         super().__init__(valeur)
27         self.nom = nom

```

Gestion des exceptions

```

1  try:
2      resultat = 10 / 0
3  except ZeroDivisionError as e:
4      print("Division par z ro !", e)
5  except Exception as e:
6      print("Autre erreur :", e)
7  else:
8      print("Pas d'erreur")
9  finally:
10     print("Ex cut dans tous les cas")
11
12 # Lever une exception
13 raise ValueError("Message d'erreur")

```

Entrées/Sorties

```

1  # Lecture
2  nom = input("Entrez votre nom : ")
3
4  # Fichiers
5  with open("fichier.txt", "r") as f:
6      contenu = f.read()          # tout le
7                                  fichier
8      lignes = f.readlines()      # liste de
9                                  lignes
10
11  with open("sortie.txt", "w") as f:
12      f.write("Hello\n")
13      f.writelines(["ligne1\n", "ligne2\n"])

```

Modules utiles

```

1  import random
2  random.random()                # float dans [0,1)

```

```

3 random.randint(1, 10)      # entier entre 1 et
   10
4 random.choice(liste)      # lment
   alatoire
5 random.shuffle(liste)     # m lange en place
6
7 import time
8 time.time()               # timestamp Unix
9 time.sleep(2)             # pause de 2
   secondes
10
11 import sys
12 sys.argv                 # arguments ligne
   de commande
13 sys.setrecursionlimit(2000)
14
15 import os
16 os.listdir(".")          # liste fichiers/
   rpertoires
17 os.path.join("dir", "fichier.txt")

```

Compréhensions et générateurs

```

1 # Liste
2 carres = [x**2 for x in range(10) if x % 2
   == 0]
3
4 # Dictionnaire
5 d = {x: x**2 for x in range(5)}
6
7 # Ensemble
8 s = {x for x in "abracadabra" if x not in "
   abc"}
9
10 # G n rateur ( valuation paresseuse)
11 gen = (x**2 for x in range(10))
12 next(gen)    # 0
13 next(gen)    # 1
14
15 # Fonction g n ratrice
16 def fibonacci_gen(n):
17     a, b = 0, 1
18     for _ in range(n):
19         yield a
20         a, b = b, a + b

```

Fonctions utiles pour les itérables

```

1 map(fonction, iterable)    # applique
   fonction
2 filter(predicat, iterable) # garde si
   predicat vrai
3 zip(it1, it2)              # combine
   lment          lment
4 enumerate(iterable)        # (index,
   lment )
5
6 # Exemple
7 list(map(lambda x: x**2, [1,2,3])) #
   [1,4,9]
8 list(filter(lambda x: x>0, [-1,0,2])) # [2]
9 list(zip([1,2], ['a','b']))        #
   [(1,'a'),(2,'b')]

```

Formatage de chaînes (f-strings)

```

1 nom = "Alice"
2 age = 25
3 print(f"{nom} a {age} ans.")
4 # Expressions
5 print(f"Dans 5 ans, {nom} aura {age+5} ans."
   )
6 # Formatage num rique
7 pi = 3.14159265
8 print(f"          {pi:.2f}")          # 3.14
9 print(f"{42:04d}")                    # 0042
10 print(f"{0.25:.1%}")                  # 25.0%

```

Tranches (slicing)

```

1 liste = [0,1,2,3,4,5]
2 liste[2:5]    # [2,3,4] (d but inclus,
   fin exclue)
3 liste[:3]     # [0,1,2]
4 liste[3:]     # [3,4,5]
5 liste[::2]    # [0,2,4] (pas de 2)
6 liste[::-1]   # [5,4,3,2,1,0] (invers e)
7
8 chaine = "Python"
9 chaine[1:4]   # "yth"

```

Copies

```

1 import copy
2 # Copie superficielle (attention aux objets
   imbriqués)
3 liste2 = liste.copy()          # ou liste[:]
4 dico2 = dico.copy()
5
6 # Copie profonde
7 liste3 = copy.deepcopy(liste)

```

Décorateurs (notion avancée)

```

1 def mon_decorateur(fonction):
2     def wrapper(*args, **kwargs):
3         print("Avant l'appel")
4         res = fonction(*args, **kwargs)
5         print("Apr s l'appel")
6         return res
7     return wrapper
8
9 @mon_decorateur
10 def dire_bonjour(nom):
11     print(f"Bonjour {nom}!")
12
13 #          : dire_bonjour =
   mon_decorateur(dire_bonjour)

```

Mesure de performance

```

1 import timeit
2
3 # Mesurer une expression
4 temps = timeit.timeit(
5     "sum(range(1000))",
6     number=10000
7 )
8 print(f"Temps moyen : {temps/10000:.6f} s")
9
10 # Avec une fonction d finie
11 def mon_algo(n):
12     return sum(i**2 for i in range(n))
13
14 temps = timeit.timeit(
15     lambda: mon_algo(1000),

```

```
16     number=1000
17 )
```

Aide et documentation

```
1 help(print)           # documentation d'une
2                        fonction
```

```
2 dir(list)              # attributs et
3                          methodes d'un objet
4 type(variable)         # type de la variable
5 isinstance(x, int)     # verification de type
```